

백그라운드 작업 관리

백그라운드 작업?

- 네트워크 요청 및 데이터 동기화
- 미디어 재생
- 위치 기반 서비스
- etc...

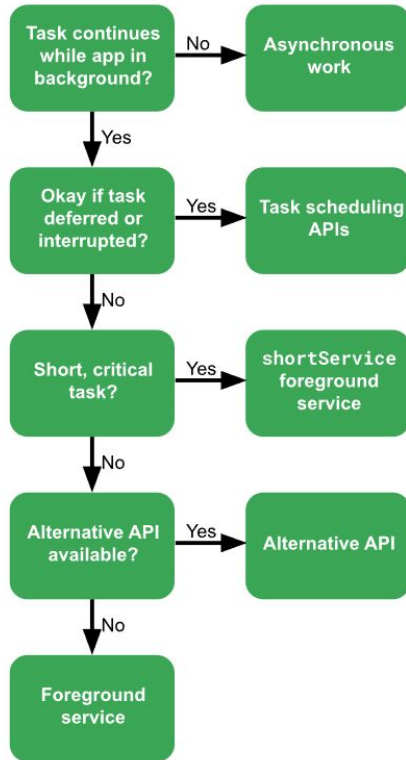


그림 1: 사용자 시작 백그라운드 작업을 실행하는 데 적합한 API를 선택하는 방법

Case 1. 앱이 사용자와 상호작용하고 있을 때

```
1 Usage
private fun fetchBottaries() {
    updateState { copy(isLoading = true) }
    fetchBottariesUseCase()
        .onEach { bottaries ->
            updateState { this: BottariUiState
                copy(
                    isLoading = false,
                    bottaries = bottaries.map { transform = BottariUiModel::fromPersonalBottari },
                    isFetched = true,
                )
            }
        }.catch { this: FlowCollector<List<PersonalBottari>> it: Throwable
            emitEvent( event = BottariUiEvent.FetchBottariesFailure)
            updateState { copy(isLoading = false) }
        }.launchIn(viewModelScope)
}
```

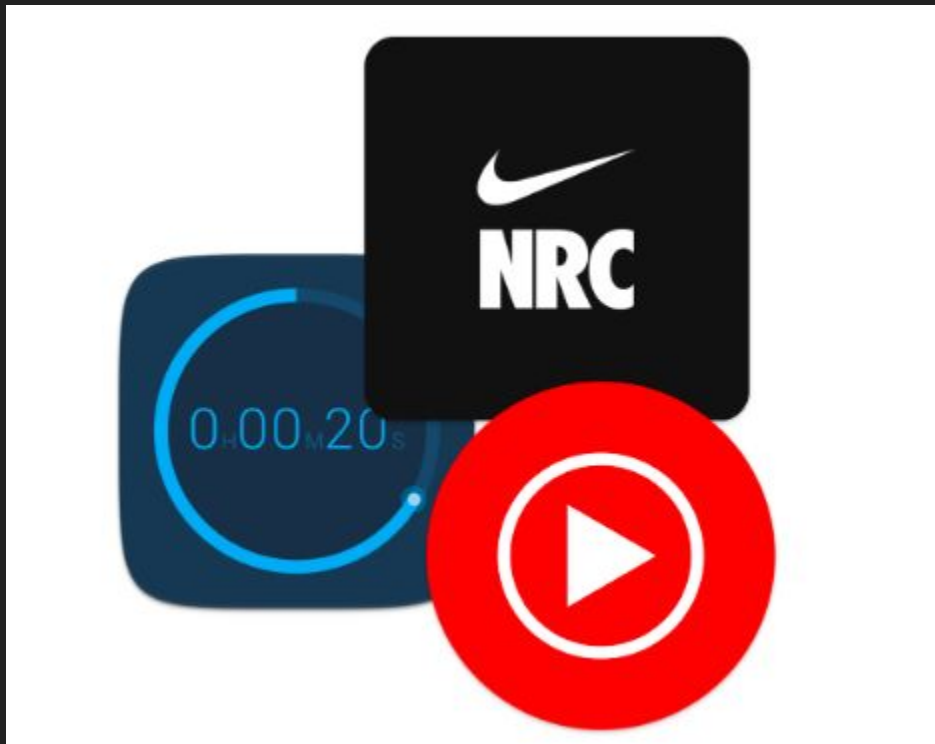
Case 2. 앱이 백그라운드에 있을 때

서비스 개요

Service는 백그라운드에서 장기 실행 작업을 실행할 수 있는 [애플리케이션 구성요소](#)입니다. 사용자 인터페이스를 제공하지 않습니다. 서비스는 일단 시작되면 사용자가 다른 애플리케이션으로 전환한 후에도 얼마 동안 계속 실행될 수 있습니다. 또한 구성요소는 서비스에 바인딩하여 서비스와 상호작용하고 프로세스 간 통신 (IPC)을 실행할 수도 있습니다. 예를 들어 서비스는 백그라운드에서 네트워크 트랜잭션을 처리하거나, 음악을 재생하거나, 파일 I/O를 실행하거나, 콘텐츠 제공업체와 상호작용할 수 있습니다.

! **주의:** 서비스는 호스팅 프로세스의 기본 스레드에서 실행됩니다. 서비스는 자체 스레드를 만들지 **않으며** 달리 지정하지 않는 한 별도의 프로세스에서 실행되지 **않습니다**. 애플리케이션 응답 없음 (ANR) 오류를 방지하려면 서비스 내에서 별도의 스레드에서 차단 작업을 실행해야 합니다.

Case 2. 앱이 백그라운드에 있을 때



Case 3. 앱이 실행되고 있지 않을 때

사용자 시작 데이터 전송

시간이 오래 걸릴 수 있는 데이터 전송을 실행해야 하는 경우 [JobScheduler](#) 작업을 만들고 이를 *사용자 시작 데이터 전송 (UIDT)* 작업으로 식별하면 됩니다. UIDT 작업은 원격 서버에서 파일을 다운로드하는 등 기기 사용자가 시작하는 장기 데이터 전송을 위한 것입니다. UIDT 작업은 Android 14 (API 수준 34)에서 도입되었습니다.

사용자 시작 데이터 전송 작업은 사용자가 시작합니다. 이러한 작업은 알림이 필요하고, 즉시 시작되며, 시스템 조건에서 허용하는 한 장시간 실행될 수 있습니다. 사용자가 시작한 데이터 전송 작업 여러 개를 동시에 실행할 수 있습니다.

사용자가 시작한 작업은 애플리케이션이 사용자에게 표시되는 동안 (또는 [허용된 조건](#) 중 하나로) 예약되어야 합니다. 모든 제약 조건이 충족되면 시스템 상태 제한에 따라 OS에서 사용자가 시작한 작업을 실행할 수 있습니다. 또한 시스템은 제공된 예상 페이로드 크기를 사용하여 작업 실행 시간을 판단할 수도 있습니다.

Case 3. 앱이 실행되고 있지 않을 때

```
class MyJobService : JobService()
    override fun onStartJob(params: JobParameters?): Boolean {
        // 백그라운드 작업 시작
        // 예시: 서버 통신 코드

        // 작업이 완료되었음을 명시해야 함
        jobFinished(params, false)

        // 작업이 비동기로 진행됨을 알림
        return true
    }

    override fun onStopJob(params: JobParameters?): Boolean {
        // 작업이 중단될 때 호출됨
        return false
    }
}
```


Case 3. 앱이 실행되고 있지 않을 때

```
import android.content.ComponentName
import android.content.Context
import android.os.Build
import androidx.annotation.RequiresApi
import androidx.work.JobInfo
import androidx.work.JobScheduler

@RequiresApi(Build.VERSION_CODES.LOLLIPOP)
fun scheduleJob(context: Context) {
    val jobInfo = JobInfo.Builder(1, ComponentName(context, MyJobService::class.java))
        .setRequiredNetworkType(JobInfo.NETWORK_TYPE_UNMETERED) // 와이파이 연결 시
        .setRequiresCharging(true) // 충전 중일 때
        .setPersisted(true) // 재부팅 후에도 유지
        .setPeriodic(15 * 60 * 1000) // 15분 마다
        .build()

    jobScheduler.schedule(jobInfo)
}
```

Case 3. 앱이 실행되고 있지 않을 때

알람 예약

알람 ([AlarmManager](#) 클래스 기반)을 사용하면 애플리케이션이 사용되지 않을 때 시간 기반 작업을 실행할 수 있습니다. 예를 들어, 알람을 사용하여 일기예보를 다운로드하는 것과 같이 하루에 한 번 서비스를 시작하는 장기 실행 작업을 시작할 수 있습니다.

알람에는 다음 특징이 있습니다.

- 지정된 시간에 또는 정해진 간격으로 인텐트를 실행합니다.
- 브로드캐스트 리시버와 함께 알람을 사용하여 [작업](#) 또는 [WorkRequest](#)를 예약하여 다른 작업을 실행할 수 있습니다.
- 애플리케이션 외부에서 작동하므로 알람을 사용하면 앱이 실행 중이 아니거나 기기가 대기 상태인 경우에도 이벤트나 작업을 트리거할 수 있습니다.
- 알람은 앱의 리소스 요구사항을 최소화하는 데 도움이 됩니다. 타이머나 지속적으로 실행되는 서비스를 사용하지 않고 작업을 예약할 수 있습니다.

★ **참고:** 애플리케이션이 실행 중인 동안 발생하는 시간 기반 작업은 [Timer](#) 및 [Thread](#)와 함께 [Handler](#) 클래스 사용을 고려해보세요. 이 접근 방식을 사용하면 Android가 시스템 리소스를 더 효과적으로 제어할 수 있습니다.

Case 3. 앱이 실행되고 있지 않을 때

2. Uses

```
private val manager: AlarmManager =  
    ApplicationContextProvider.applicationContext.getSystemService(serviceClass = AlarmManager::class.java)  
  
private fun scheduleAlarmInternal(  
    context: Context,  
    notification: NotificationUiModel,  
    triggerTime: Long,  
) {  
    val editPendingIntent = createEditPendingIntent(context, notification)  
    val pendingIntent = createPendingIntent(context, notification)  
    val alarmClockInfo = AlarmClockInfo(triggerTime, showIntent = editPendingIntent)  
    manager.setAlarmClock(alarmClockInfo, operation = pendingIntent)  
}
```

Case 3. 앱이 실행되고 있지 않을 때

```
class AlarmReceiver : BroadcastReceiver() {  
    1 Usage  
    private val notificationHelper: NotificationHelper by lazy { NotificationHelper() }  
  
    override fun onReceive(  
        context: Context?,  
        intent: Intent,  
    ) {  
        val notification = intent.getParcelableCompat<NotificationUiModel>(key = EXTRA_NOTIFICATION)  
        notificationHelper.sendPersonalNotification(bottariId = notification.bottariId, bottariTitle = notification.bottariTitle)  
        scheduleNextAlarm(notification = notification)  
        BottariLogger.vi(  
            type = UiEventType.NOTIFICATION_CREATE,  
            params = mapOf("notification_id" to notification.bottariId, "time" to LocalDateTime.now().toString()),  
        )  
    }  
}
```

Case 3. 앱이 실행되고 있지 않을 때

시스템 리소스 소비

시스템이 앱에서 설정한 정확한 알람을 트리거하면 기기는 배터리 수명과 같은 리소스를 상당히 많이 소비합니다. 특히 절전 모드인 경우에 그렇습니다. 또한 시스템은 리소스를 더 효율적으로 사용하려고 이러한 요청을 쉽게 일괄 처리할 수 없습니다.

가능하면 언제든지 정확하지 않은 알람을 만드는 것이 좋습니다. 더 긴 작업을 실행하려면 알람의 `BroadcastReceiver` 에서 `WorkManager` 또는 `JobScheduler` 를 사용하여 예약하세요. 기기가 잠자기 모드일 때 작업을 실행하려면 `setAndAllowWhileIdle()` 을 사용하여 부정확한 알람을 만들고 알람에서 작업을 시작합니다.

Case 3. 앱이 실행되고 있지 않을 때

태스크 예약

앱이 표시 상태를 벗어나도 계속 실행되는 작업을 실행하려면 Jetpack 라이브러리 [WorkManager](#)를 사용하는 것이 좋습니다. WorkManager는 앱 다시 시작 및 기기 재부팅 시 작업을 유지할 수 있는 강력한 예약 메커니즘을 제공합니다.

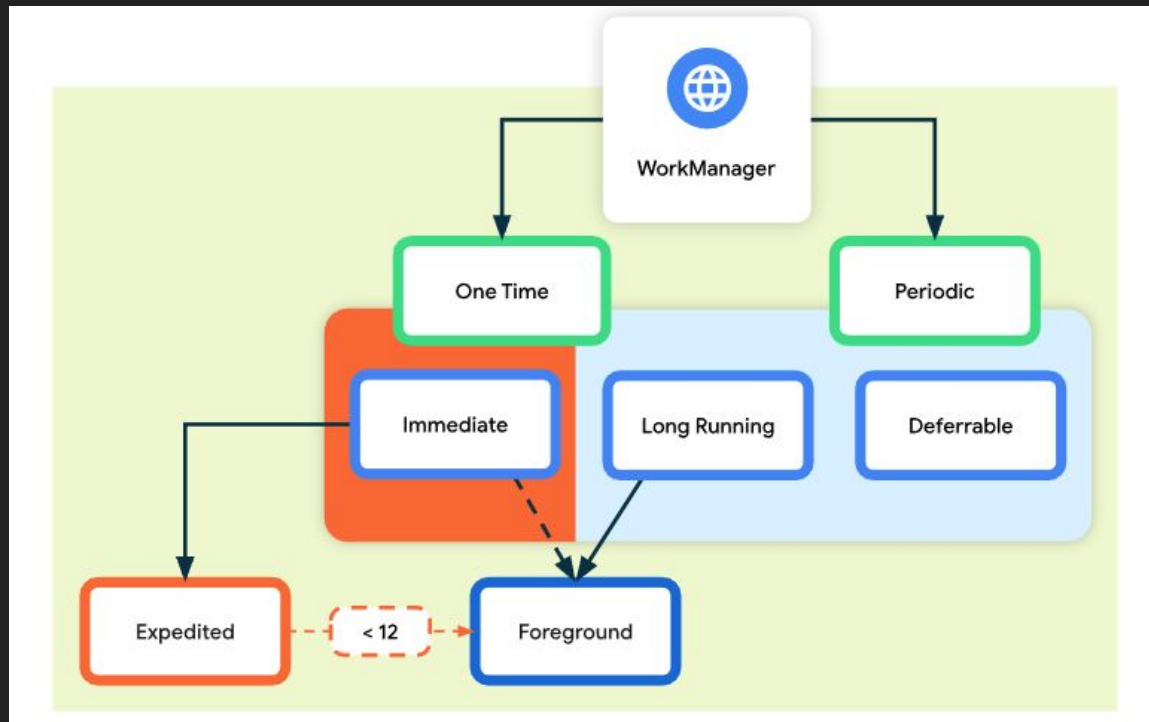
작업 유형

WorkManager는 세 가지 유형의 작업을 처리합니다.

- **즉시:** 즉시 시작하고 곧 완료해야 하는 작업입니다. 신속하게 처리될 수 있습니다.
- **장기 실행:** 더 오래(10분 이상이 될 수 있음) 실행될 수 있는 작업입니다.
- **지연 가능:** 나중에 시작하며 주기적으로 실행될 수 있는 예약된 작업입니다.

그림 1은 다양한 유형의 작업이 서로 어떻게 관련되어 있는지 보여줍니다.

Case 3. 앱이 실행되고 있지 않을 때



Case 3. 앱이 실행되고 있지 않을 때

AlarmManager + JobScheduler + ForegroundService + BroadcastReceiver

= WorkManager

WorkManager는 내부적으로 모두 사용하고 있다.

Case 3. 앱이 실행되고 있지 않을 때

```
class NotificationWorker(
    context: Context,
    workerParams: WorkerParameters,
    private val getNotificationsUseCase: GetNotificationsUseCase,
) : CoroutineWorker( appContext = context, workerParams) {
    override suspend fun doWork(): Result =
        getNotificationsUseCase()
            .mapCatching( transform = ::scheduleActiveAlarms)
            .fold(
                onSuccess = { Result.success() },
                onFailure = { exception ->
                    BottariLogger.error( message = exception.stackTraceToString(), throwable = exception)
                    Result.retry() ^lambda
                },
            )
}

1 Usage
private fun scheduleActiveAlarms(notifications: List<Notification>) =
    notifications.forEach { notification ->
        if (notification.alarm.isActive) {
            scheduleAlarm(notification = NotificationUiModel.fromDomain(notification))
        }
    }
}
```

Case 3. 앱이 실행되고 있지 않을 때

```
class BootReceiver : BroadcastReceiver() {  
    override fun onReceive(  
        context: Context,  
        intent: Intent,  
    ) {  
        if (intent.action == Intent.ACTION_BOOT_COMPLETED) {  
            val workRequest =  
                OneTimeWorkRequestBuilder<NotificationWorker>()  
                    .setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST)  
                    .build()  
            WorkManager.getInstance(context).enqueue(workRequest)  
        }  
    }  
}
```

결론

앱이 실행되고 있을 때 -> Coroutine

백그라운드에서 있으며, 사용자에게 알려야 할 때 -> ForegroundService (WorkManager 대체 가능)

앱이 실행되고 있지 않을 때

- 특정 날짜 및 시간에 반드시 실행해야 할 때 -> AlarmManager
- 특정 조건에 따라 실행해야 할 때 -> JobScheduler (WorkManager 대체 가능)

그냥 WorkManager를 쓰면 대부분 해결된다.

안드로이드에서도 WorkManager 권장함